

Python Advanced Master Sheet

By Leon · 2025

1 Object Model & Copying

- Everything in Python is an **object** with an *id*, *type*, and *value*.
- **Assignment (=)** binds a new name to the same object (no copy).
- **Shallow copy** duplicates the container but keeps references.
- **Deep copy** recursively duplicates all nested objects.
- 📌 `copy.copy(obj)` vs `copy.deepcopy(obj)`

Tip: Immutable objects (int, str, tuple) are not affected by shallow/deep copy differences.

2 Iterators & Generators

- **Iterable** → implements `__iter__()`
- **Iterator** → implements both `__iter__()` and `__next__()`
- **Generator** → function with `yield`; pauses & resumes.
- **Generator expression:** `(x*x for x in range(5))` — lazy, memory-efficient.

Key tools:

`itertools.compress`, `itertools.starmap`, `enumerate`, `zip`

3 Functional Programming

- `zip(a, b)` → combine sequences elementwise.
- `map(func, seq)` → apply transformation.
- `filter(func, seq)` → keep truthy elements.
- `reduce(func, seq, init)` → accumulate results.

- `functools.partial()` → pre-bind arguments for new functions.

🧩 Example:

```
from functools import reduce, partial
from operator import add
reduce(add, [1,2,3], 0) # → 6
square = partial(pow, exp=2)
square(5) # 25
```

4 Decorators & Context Managers

- **Decorator:** wraps a function to add reusable logic.

```
from functools import wraps
def log(fn):
    @wraps(fn)
    def wrapper(*a, **kw):
        print("Running", fn.__name__)
        return fn(*a, **kw)
    return wrapper
```

- **Context Manager:** ensures setup & cleanup.

```
with open("data.txt") as f:
    content = f.read()
```

- Custom via `__enter__` / `__exit__` or `@contextmanager`.

5 Classes & Metaclasses

- **Class variables** shared; **instance vars** unique per object.
- `@staticmethod` → no access to class/instance.
- `@classmethod` → receives class as first arg.

- **ABC (Abstract Base Class)** defines interfaces.
 - `@register` → declares virtual subclass (recognized by `isinstance`).
 - **Metaclass** → controls how classes are created (type by default).
-

6 Dynamic Features & Introspection

- Add attributes/methods dynamically:
 - `eval(expr)` executes an expression (⚠ unsafe).
 - `exec(code)` runs statements, modifies scope.
 - `repr(obj)` returns developer-readable form.
 - Use `ast.literal_eval()` for safe evaluation.
-

7 File & Data Handling

File IO

```
with open("data.txt", "w") as f:  
    f.write("hello")
```

Data Formats

- CSV → `csv.writer`, `csv.DictWriter`
- JSON → `json.dump()` / `json.load()`
- YAML → `yaml.safe_load()`
- XML → `xml.etree.ElementTree`
- Pickle → for binary serialization (Python-only, ⚠ unsafe on untrusted data)

Hashing

```
import hashlib  
hashlib.sha256(b"hello").hexdigest()
```

8 Concurrency & Async

- **Threading** → good for I/O tasks.
 - **Multiprocessing** → good for CPU-heavy tasks.
 - **AsyncIO** → lightweight cooperative multitasking.
 - `async def` → coroutine function
 - `await` → executes coroutine
 - `asyncio.gather()` → run multiple concurrently
-

9 Multiprocessing & Synchronization

- `multiprocessing.Manager().list()` → shared state
 - `Queue` → safe communication between processes
 - `Pipe` → direct one-to-one communication
 - `Value('i', 100)` → shared scalar; use `get_lock()` to ensure atomic updates
-

10 Testing, Profiling & Deployment

- **Unit Testing:** `pytest`
 - **Coverage:** `pytest --cov`
 - **Performance:** `py-spy`, `Scalene`, `cProfile`, `timeit`
 - **Packaging:** `venv`, `poetry`, `pyinstaller`, `Docker`
-

✨ Memory Tricks (Quick Recall)

Concept	Tip
Copy	"=" rebinds; shallow = new shell; deep = new heart
Iterable	needs <code>__iter__</code> ; Iterator adds <code>__next__</code>
Partial	bind keyword args for flexibility
Sorting	<code>key=lambda p:p[1]</code> , <code>reverse=True</code>

Concept	Tip
With	enter = acquire; exit = release
ABC	register() only declares relationship
Eval	avoid external input; use literal_eval
Concurrency	I/O → Threads, CPU → Processes
Async	coroutine object idle until await
Testing	pytest + coverage + Scalene for bottlenecks